Setup your GitHub Repository and Azure access

In this section, you will create an Azure Service Principal to authorize GitHub accessing your Azure subscription from GitHub Actions. You will also setup the GitHub workflow that will build, test and deploy your website to Azure.

Create an Azure Service Principal and save it as GitHub secret

In this task, you will create the Azure Service Principal used by GitHub to deploy the desired resources. As an alternative, you could also use OpenID connect in Azure, as a secretless authentication mechanism.

1. On your lab computer, in a browser window, open the Azure Portal at https:*//portal.azure.com*.
2. In the portal, look for **Resource Groups** and select on it.
3. Select the resource group **rg-eshoponweb**
4. In the Azure Portal, open the **Cloud Shell** (next to the search bar).

Note: if the Azure portal asks you to create a storage, you can choose **No storage account required** options, select your subscription and select on **Apply** button

5. Make sure the terminal is running in **Bash** mode and execute the following command, replacing **SUBSCRIPTION-ID** and **RESOURCE-GROUP** with your own identifiers (both can be found on the **Overview** page of the Resource Group):

bashTypeCopy

az ad sp create-for-rbac --name GH-Action-eshoponweb61512893 --role contributor --scopes /subscriptions/SUBSCRIPTION-ID/resourceGroups/RESOURCE-GROUP --sdk-auth

Note: Make sure this is typed or pasted as a single line!

Note: this command will create a Service Principal with Contributor access to the Resource Group created before. This way we make sure GitHub Actions will only have the permissions needed to interact only with this Resource Group (not the rest of the subscription)

6. The command will output a JSON object, you will later use it as a GitHub secret for the workflow. Copy the JSON. The JSON contains the identifiers used to authenticate against Azure in the name of a Microsoft Entra identity (service principal).

{

"clientId": "<GUID>",

"clientSecret": "<GUID>",

"subscriptionId": "<GUID>",

"tenantId": "<GUID>",

(...)

}

7. (Skip if already registered) You also need to run the following command to register the resource provider for the **Azure App Service** you will deploy later:

az provider register --namespace Microsoft.Web

8. In a browser window, go back to your **eShopOnWeb** GitHub repository.
9. On the repository page, go to **Settings**, select on **Secrets and variables > Actions**. Select on **New repository secret**
    - Name : **AZURE_CREDENTIALS**
    - Secret: **paste the previously copied JSON object** (GitHub is able to keep multiple secrets under same name, used by azure/login action)
10. Select on **Add secret**. Now GitHub Actions will be able to reference the service principal, using the repository secret.

Modify and execute the GitHub workflow

In order to determine an **Azure** region with capacity, select a region from the dropdown below and then select the **Check Selected Region** button. Once a region is found to have capacity, you may proceed.

eastuseastus2westuswestus2westus3westeuropecanadacentralsoutheastasiaaustraliaeast

Success

Region has capacity: westeurope

In this task, you will modify the given GitHub workflow and execute it to deploy the solution in your own subscription.

1. In a browser window, go back to your **eShopOnWeb** GitHub repository.
2. On the repository page, go to **Code** and open the following file: **eShopOnWeb/.github/workflows/eshoponweb-cicd.yml**. This workflow defines the CI/CD process for the given .NET 8 website code.
3. Uncomment the **on** section (delete "#"). The workflow triggers with every push to the main branch and also offers manual triggering ("workflow_dispatch").
4. In the **env** section, make the following changes:

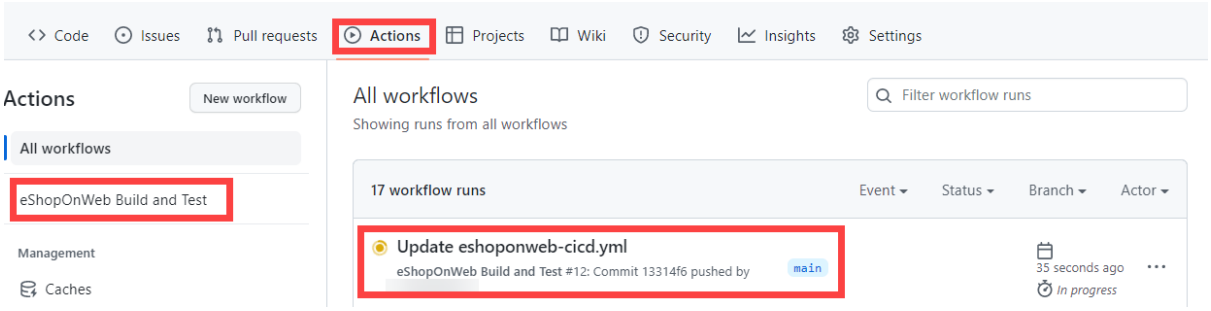
- Replace **rg-eshoponweb-NAME** in **RESOURCE-GROUP** variable. It should be the resource group rg-eshoponweb
- For **LOCATION** use westeurope
- Replace **YOUR-SUBS-ID** in **SUBSCRIPTION-ID** with 605ef48f-55de-418a-8839-c4d452fd22e3.
- Replace **NAME** in **WEBAPP-NAME** with eshoponweb-webapp61512893. It will be used to create a globally unique website using Azure App Service.

5. Read the workflow carefully, comments are provided to help understand.
6. Select on **Commit changes…** on top right and **Commit changes** leaving defaults (changing the main branch). The workflow will get automatically executed.

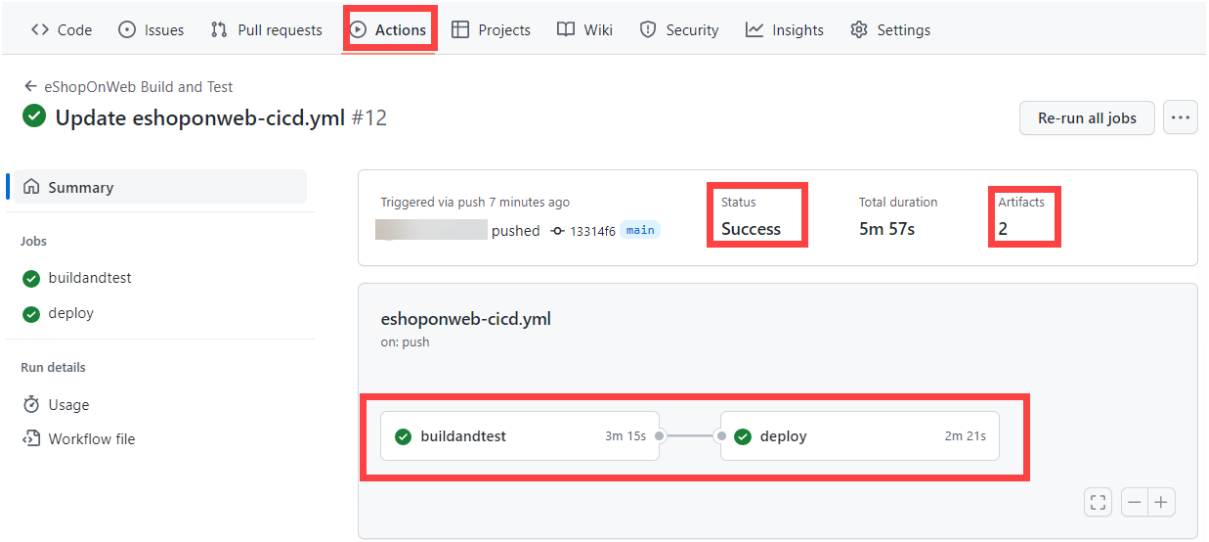
Review GitHub Workflow execution

In this task, you will review the GitHub workflow execution:

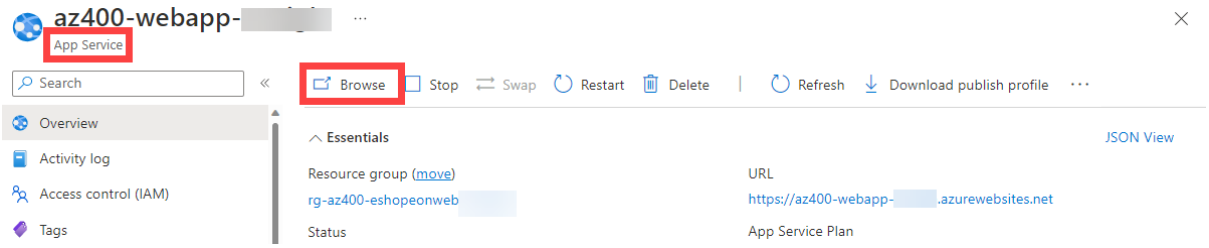
1. In a browser window, go back to your **eShopOnWeb** GitHub repository.
2. On the repository page, go to **Actions**, you will see the workflow setup before executing. Select on it.



3. Wait for the workflow to finish. From the **Summary** you can see the two workflow jobs, the status and Artifacts retained from the execution. You can select in each job to review logs.



4. In a browser window, go back to the Azure Portal at https://*portal.azure.com*. Open the resource group created before. You will see that the GitHub Action, using a bicep template, has created an Azure App Service Plan + App Service. You can see the published website opening the App Service and selecting **Browse**.



(OPTIONAL) Add manual approval pre-deploy using GitHub Environments

In this task, you will use GitHub environments to ask for manual approval before executing the actions defined on the deploy job of your workflow.

1. On the repository page, go to **Code** and open the following file: **eShopOnWeb/.github/workflows/eshoponweb-cicd.yml**.

2. In the **deploy** job section, you can find a reference to an **environment** called **Development**. GitHub used environments add protection rules (and secrets) for your targets.

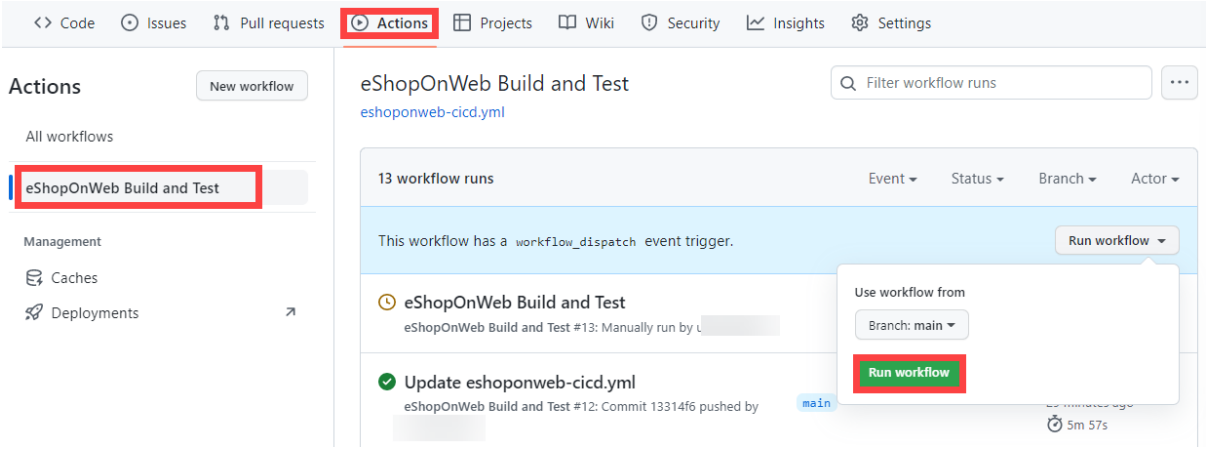
3. On the repository page, go to **Settings**, open **Environments** and select **New environment**.

4. Give it **Development** name and select on **Configure Environment**.

Note: If an environment called **Development** already exists in the **Environments** list, open its configuration by selecting on the environment name.

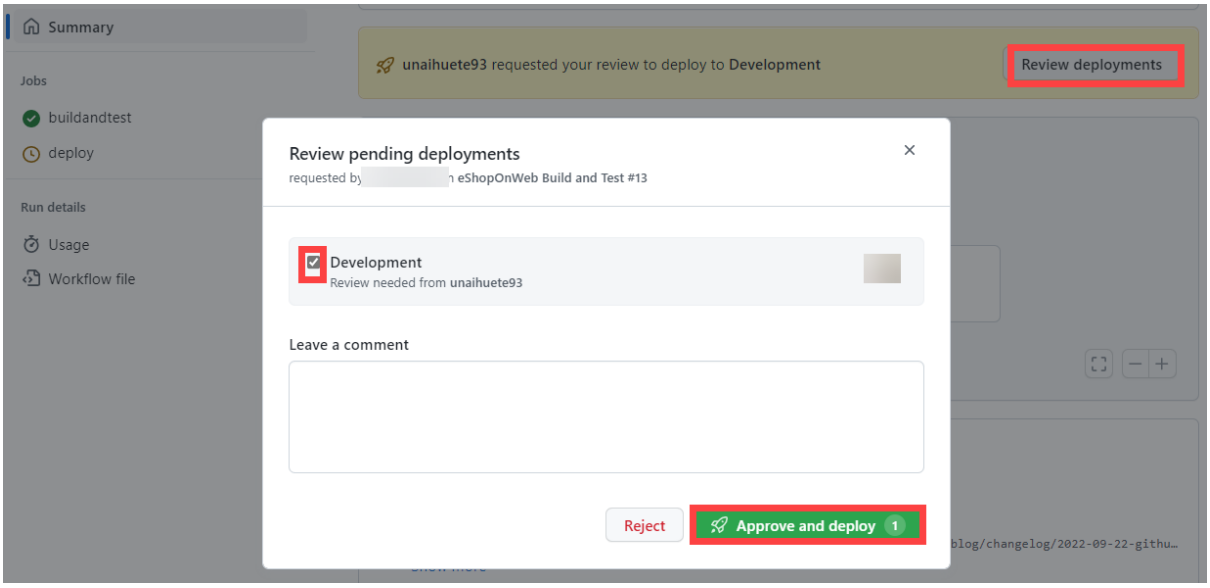
5. In the **Configure Development** tab, check the option **Required Reviewers** and your GitHub account as a reviewer. Select on **Save protection rules**.

6. Now lets test the protection rule. On the repository page, go to **Actions**, select on **eShopOnWeb Build and Test** workflow and select on **Run workflow > Run workflow** to execute manually.



7. Select on the started execution of the workflow and wait for **buildandtest** job to finish. You will see a review request when **deploy** job is reached.

8. Select on **Review deployments**, check **Development** and select on **Approve and deploy**.



9. Workflow will follow the **deploy** job execution and finish.

Clean up resources

When you complete the lab, it's important to clean up your Azure resources to avoid unnecessary charges:

Delete the Azure resources

1. In the Azure Portal at https:*//portal.azure.com*, navigate to the **Resource groups** section.
2. Find and select the **rg-eshoponweb-NAME** resource group you created.
3. On the resource group page, select **Delete resource group**.
4. Type the resource group name to confirm deletion and select **Delete**.
5. Wait for the deletion process to complete.

Clean up GitHub resources (optional)

If you want to clean up your GitHub repository:

1. Navigate to your **eShopOnWeb** repository on GitHub.
2. Go to **Settings** and scroll down to the **Danger Zone**.
3. Select **Delete this repository** and follow the prompts to confirm deletion.

CAUTION: Deleting the repository will permanently remove all code, issues, pull requests, and other repository data.

IMPORTANT: Remember to delete the Azure resources to avoid unnecessary charges. The GitHub repository can remain as part of your portfolio.